

AI Lab # 7

This lab explores adversarial search strategies by implementing the Minimax algorithm and Alpha-Beta pruning to determine optimal moves in two-player competitive games.

Artificial Intelligence (LAB)

Instructor: Mubbashir Ahmed

Email: mubbashir.ahmed@iqra.edu.pk

Date: 16 August, 2026

Today's Agenda

01

What is Minimax Algorithm?

02

Algorithm Steps and Example

03

What is Alpha-Beta Pruning?

04

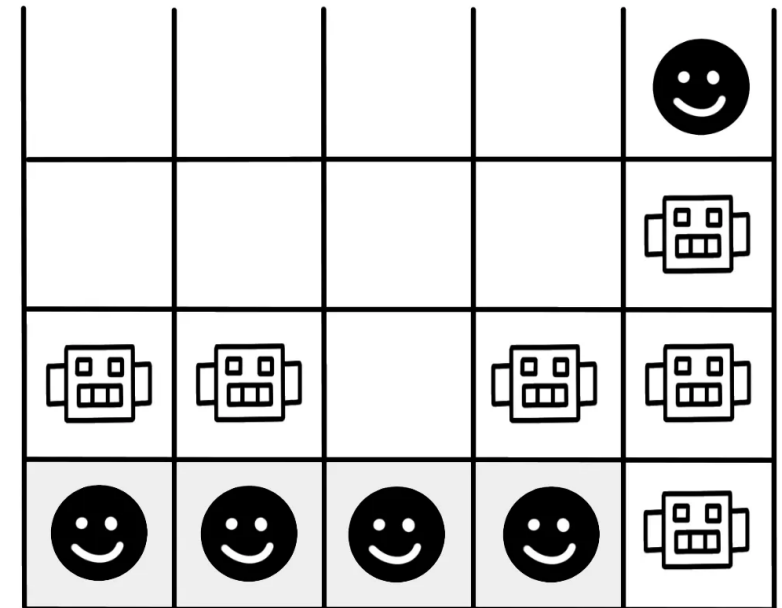
Algorithm Steps and Example

05

Lab Tasks

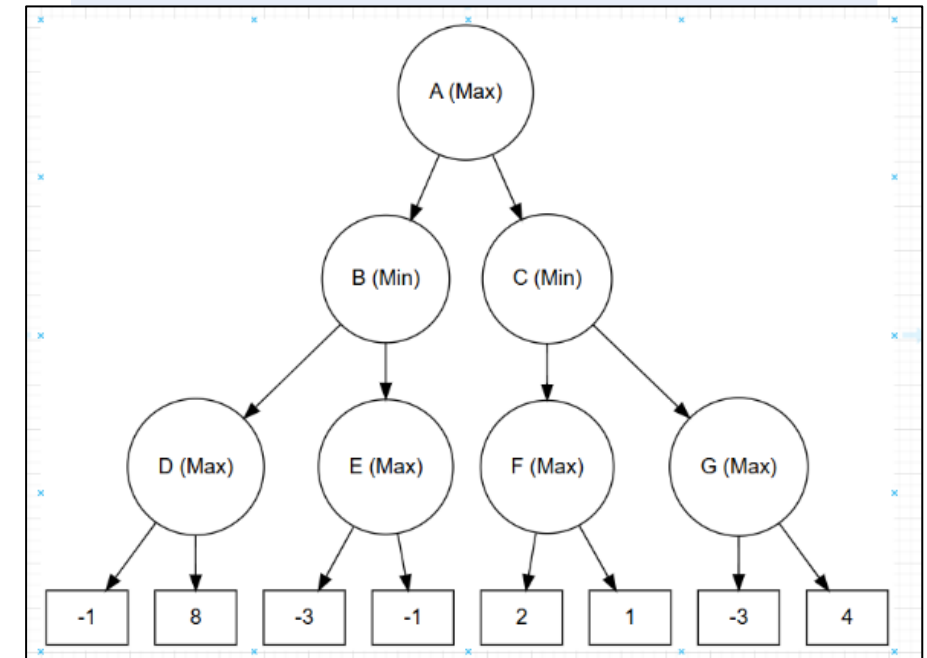
Adversarial Search

- Used in games where two players play against each other and want to win.
- These algorithms help a player decide the best move, while thinking that the other player will also try their best to win.
- For example, in games like chess or tic-tac-toe, if one player wins, the other player loses.



What is Minimax Algorithm?

- **Max nodes** represent the current player's move, aiming to maximize their advantage.
- **Min nodes** represent the opponent's move, aiming to minimize the current player's advantage.
- Max is human and min is machine(opponent)
- Max(I) will try to maximize its utility; means I will make my move best so I will get more points/reward.
- Min(machine) will try to minimize my utility, by choosing the worst move. Worst according to me not according to machine. Best according to machine
- Minimax helps you think ahead: “If I do this, what will my friend do next? And if my friend does that, what will I do?”



Algorithm Steps

1

Label the tree levels: Starting from the root, mark the first level Max, the next level Min, then Max again — alternating until there are no more child nodes left.

2

Traverse depth-first, left to right: Go as deep as possible down the leftmost branch first. At each leaf node, take its given value directly (no calculation needed).

3

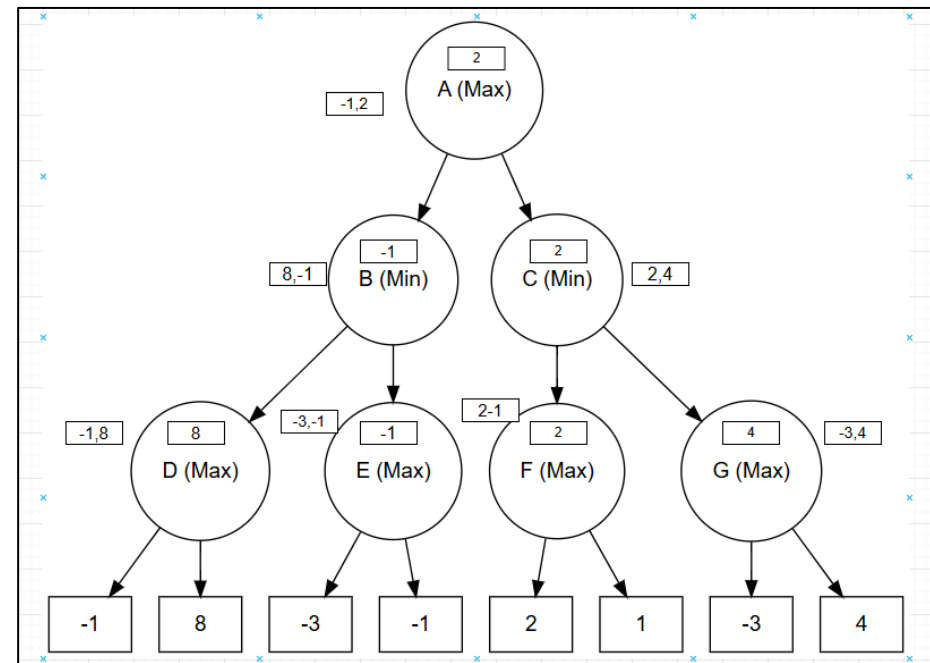
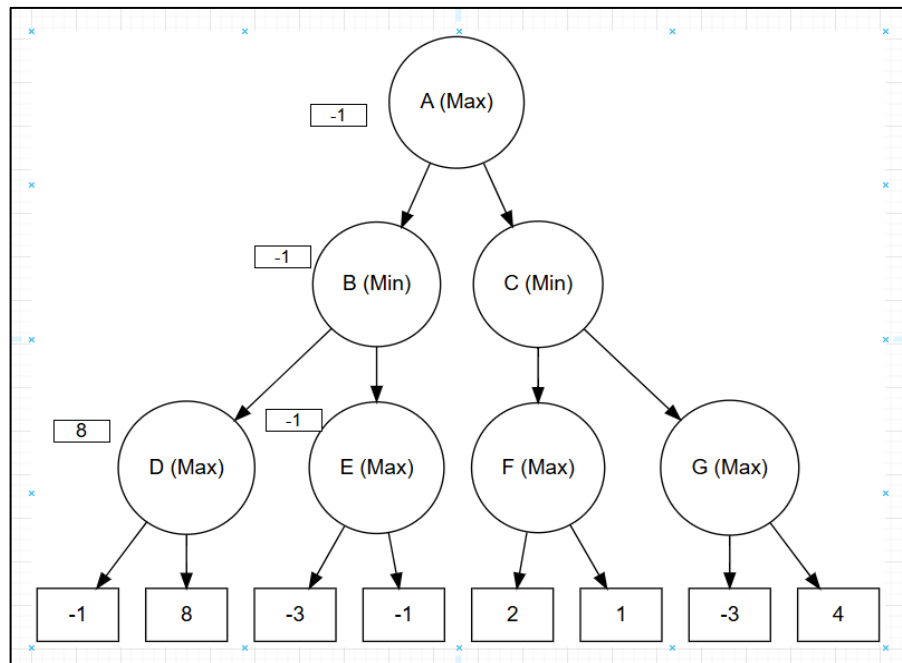
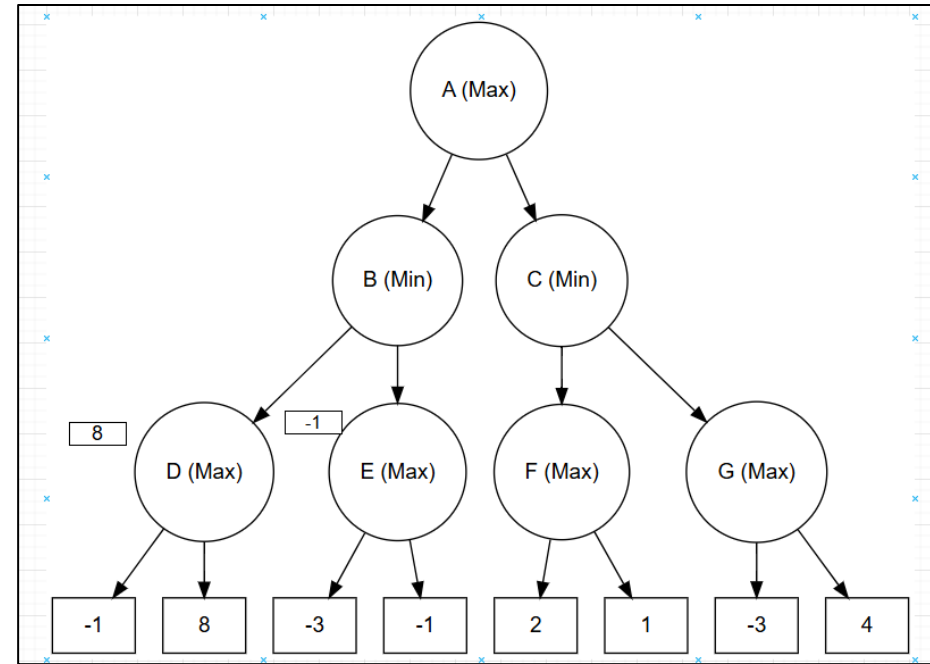
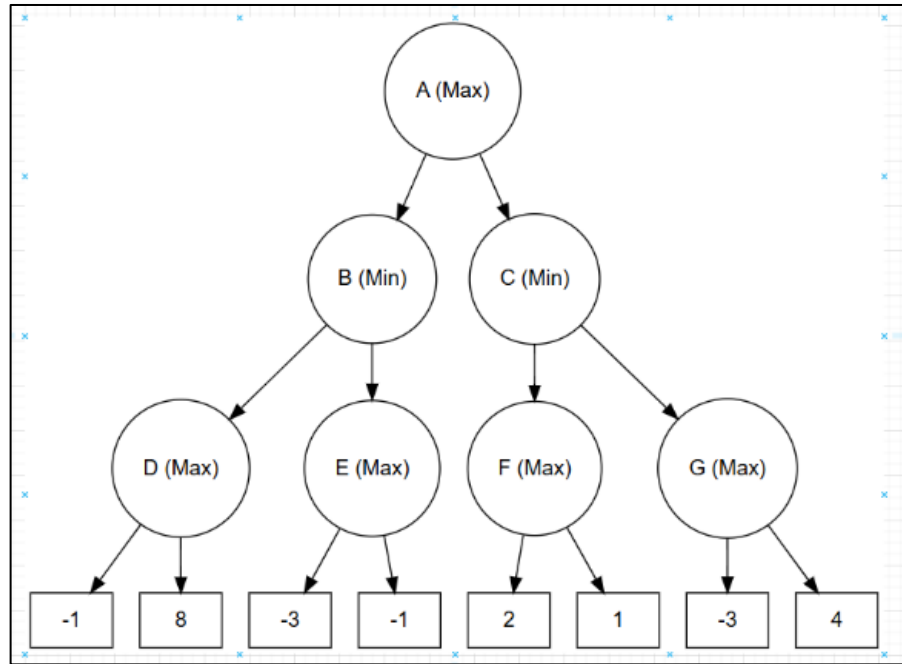
Evaluate at Min nodes: At a Min node, compare the values of its children and select the smallest one (since Min wants to minimize your outcome).

4

Evaluate at Max nodes: At a Max node, compare the values of its children and select the largest one, then pass that value up to its parent.

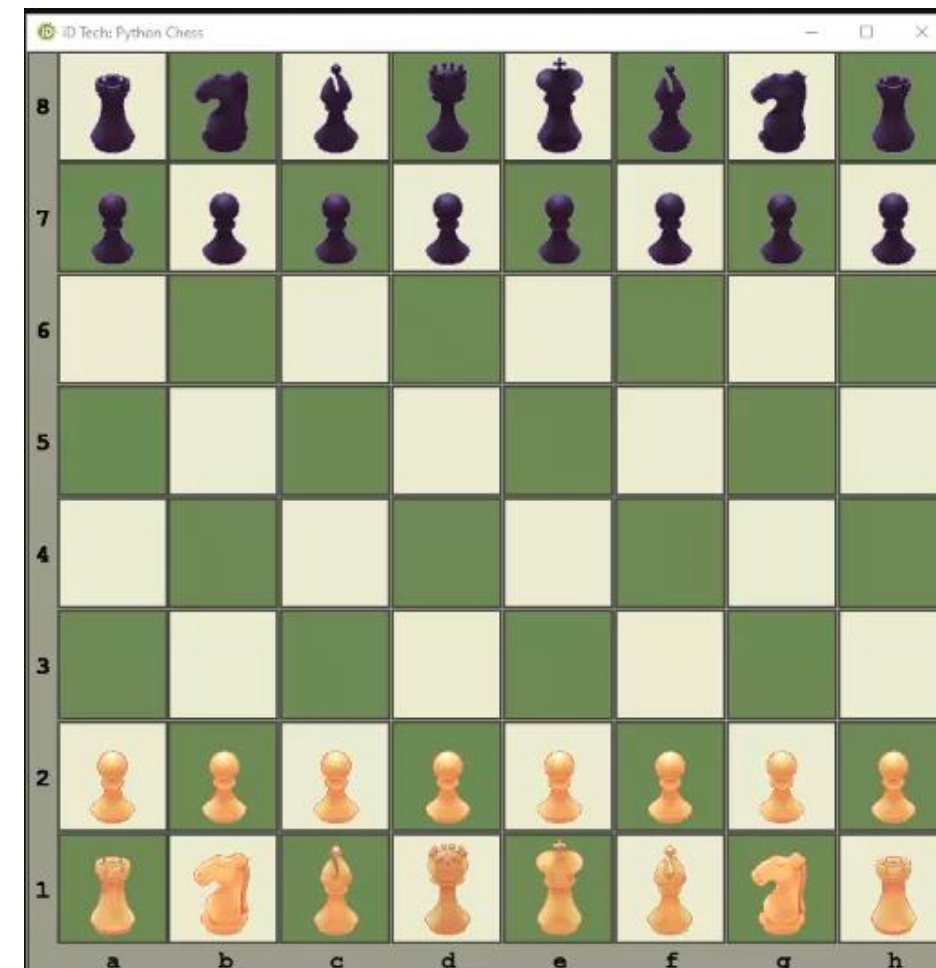
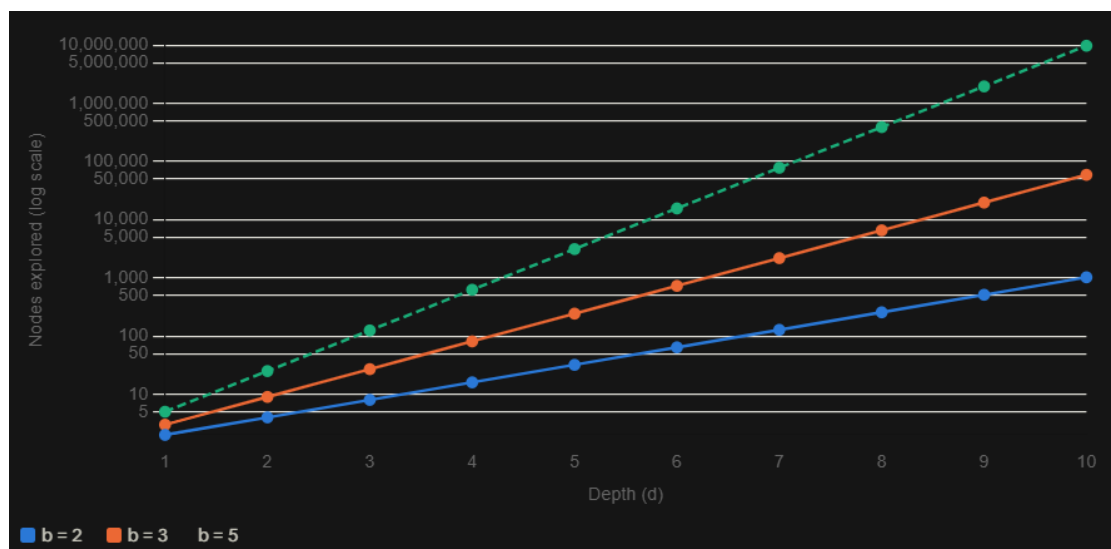
5

Repeat for all branches and finalize at root: Repeat steps 2–4 until every node has a backed-up value. The value that reaches the root (A) represents the optimal outcome, and the path leading to it is the best move sequence.



Limitations of Minimax Algorithm

- The Minimax algorithm is smart, but it can be very slow.
- It has to check a lot of moves, especially in big games like chess.
- In computer terms, its speed is written as $O(b^d)$, which means the deeper it thinks, the number of moves it checks grows very fast.
- Because of this, it cannot think very far unless we use tricks like Alpha-Beta pruning to skip moves that are not important.



Example of Minimax Algorithm

Now that we've covered Minimax, let's move towards practical code-based example

Minimax Algorithm

Minimax on nodes A, B, C, D, E, F, G. Starting node is A.



```
tree = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'] }
values = { 'D': [-1, 8], 'E': [-3, -1], 'F': [2, 1], 'G': [-3, 4] }

def minimax(node, is_max_turn):
    if node in values:
        if is_max_turn:
            return max(values[node]) # Max wants the biggest number
        else:
            return min(values[node]) # Min wants the smallest number
    if is_max_turn:
        best = -10
        for child in tree[node]:
            score = minimax(child, False)
            if score > best:
                best = score
        return best
    else:
        best = 10
        for child in tree[node]:
            score = minimax(child, True)
            if score < best:
                best = score
        return best

best_score = minimax('A', True)
print("Max's best score is:", best_score)
```

What is Alpha-Beta Pruning?

- Alpha-beta pruning is an optimization technique for the minimax algorithm.
- Alpha-beta pruning is a trick that helps you skip looking at moves/nodes that won't help you win anyway.
- It's like crossing out bad choices so you can focus on the best ones faster.

While checking moves, you keep two numbers:

- **Alpha (α):** Your best score so far ("max").
- **Beta (β):** Your opponent's best score so far ("min").

Where it is used?

- In computer games like Chess or Tic-Tac-Toe.
- In robots that play games.
- Anywhere two players or teams try to win against each other.

Algorithm Steps

1

Label the tree levels: Starting from the root, mark the first level Max, the next level Min, then Max again — alternating until there are no more child nodes left.

2

Initialize alpha and beta: Start with $\alpha = -\infty$ (best score Max can guarantee so far) and $\beta = +\infty$ (best score Min can guarantee so far). Pass these values down as you traverse.

3

Traverse depth-first, left to right: Go as deep as possible down the leftmost branch first. At each leaf node, take its given value directly and return it upward.

4

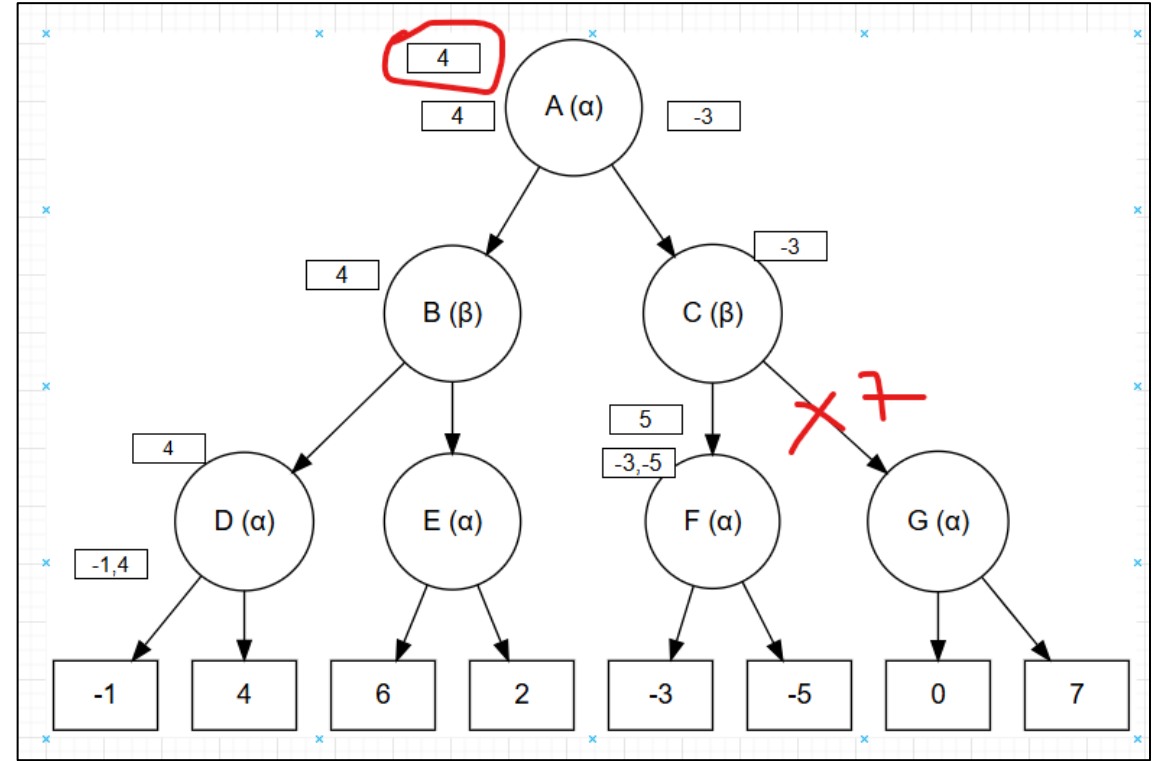
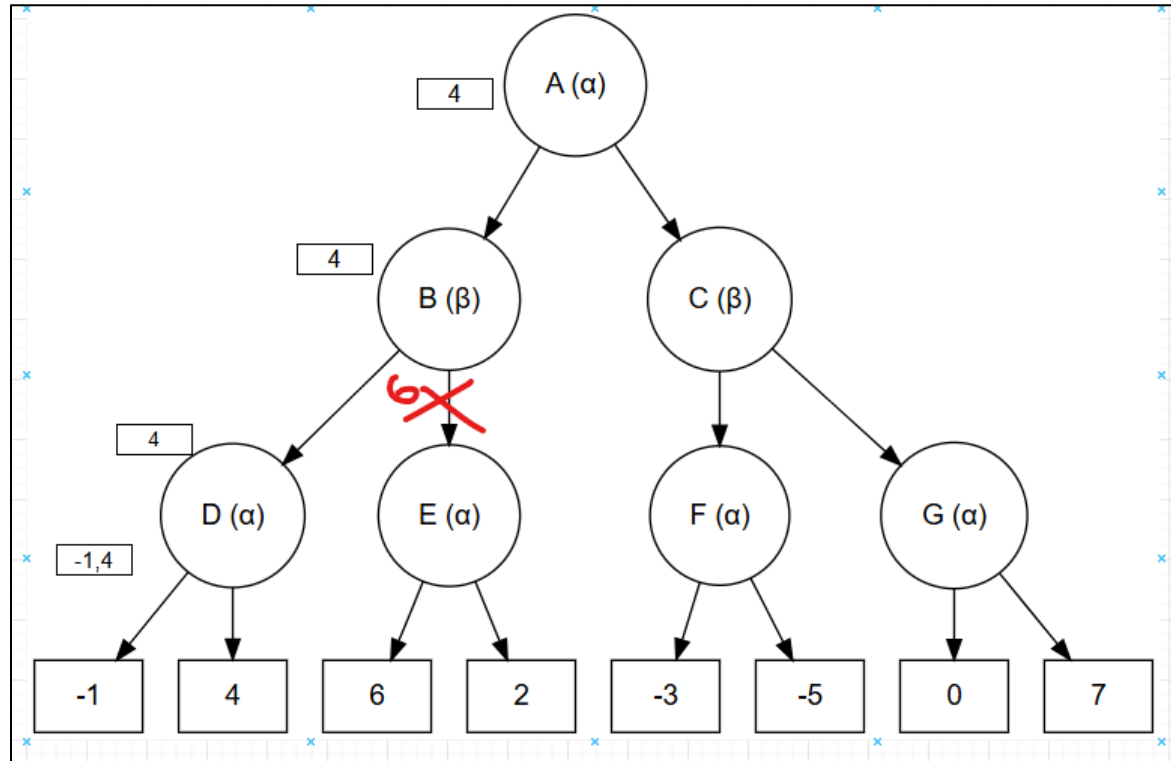
Update alpha/beta at each node: At a Max node, update alpha if a child's returned value is higher than the current alpha. At a Min node, update beta if a child's returned value is lower than the current beta.

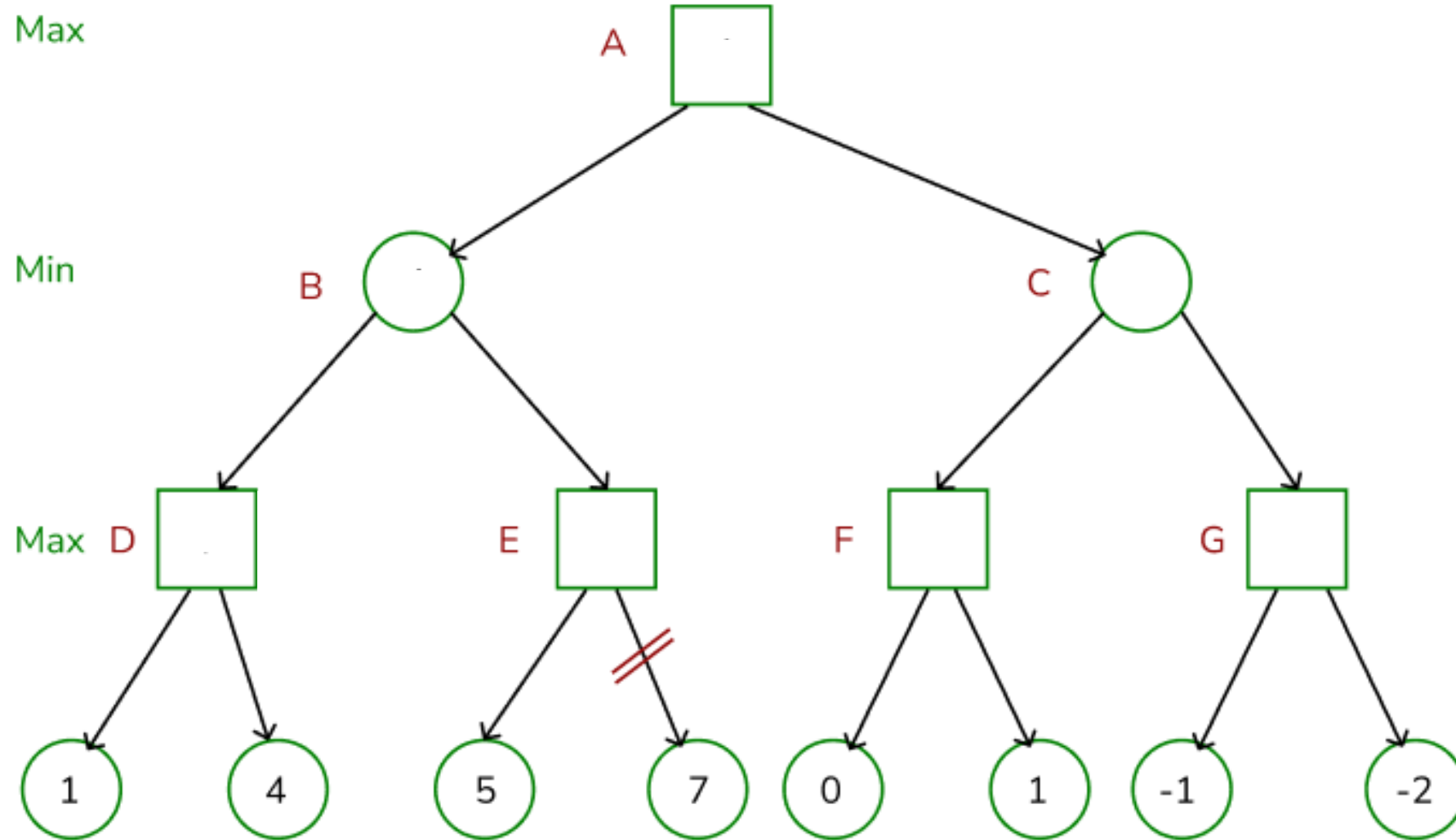
5

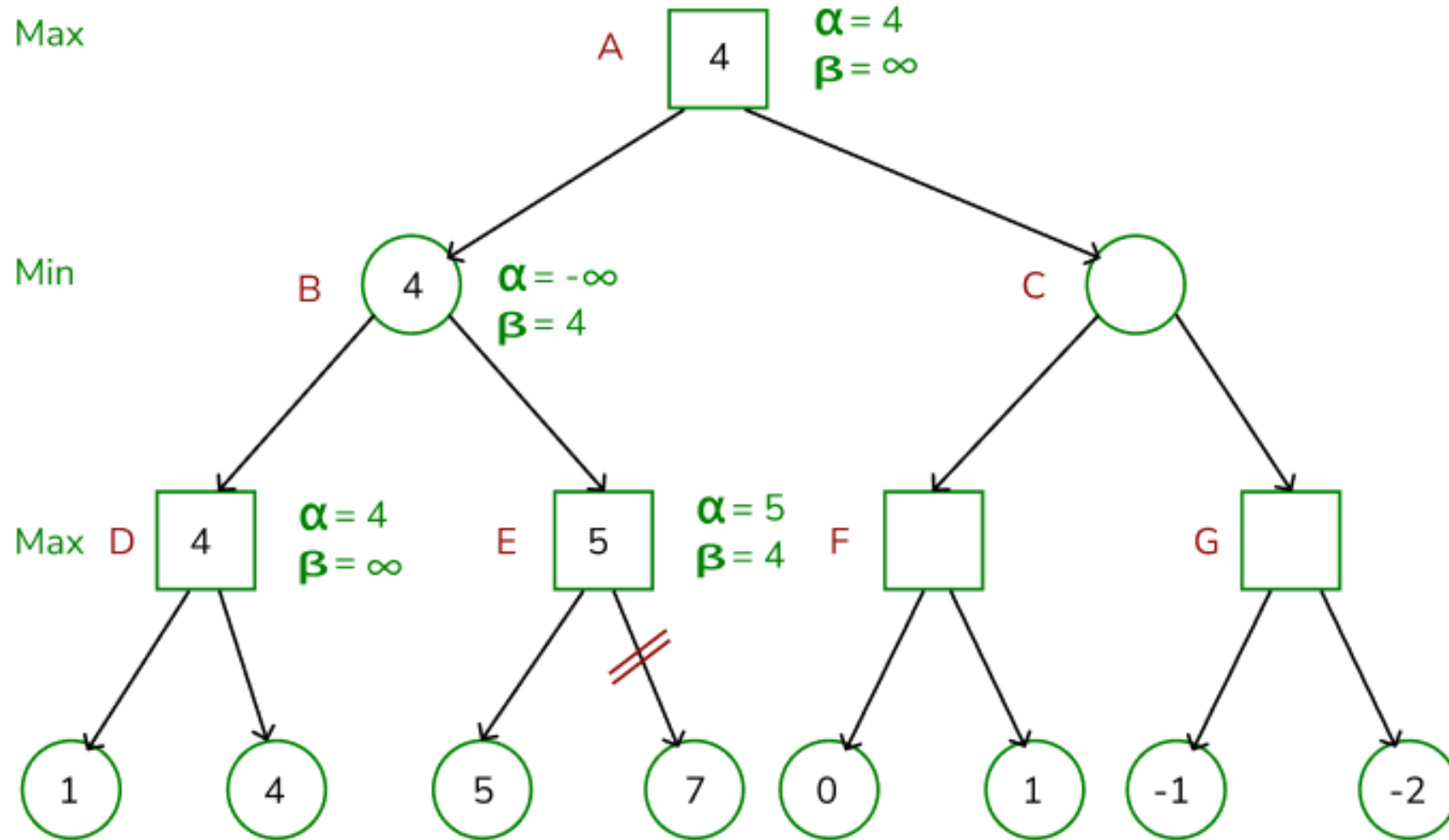
Prune when $\beta \leq \alpha$: After updating alpha or beta, check if $\beta \leq \alpha$. If true, stop exploring the remaining children of that node. Skip straight to returning the best value found so far.

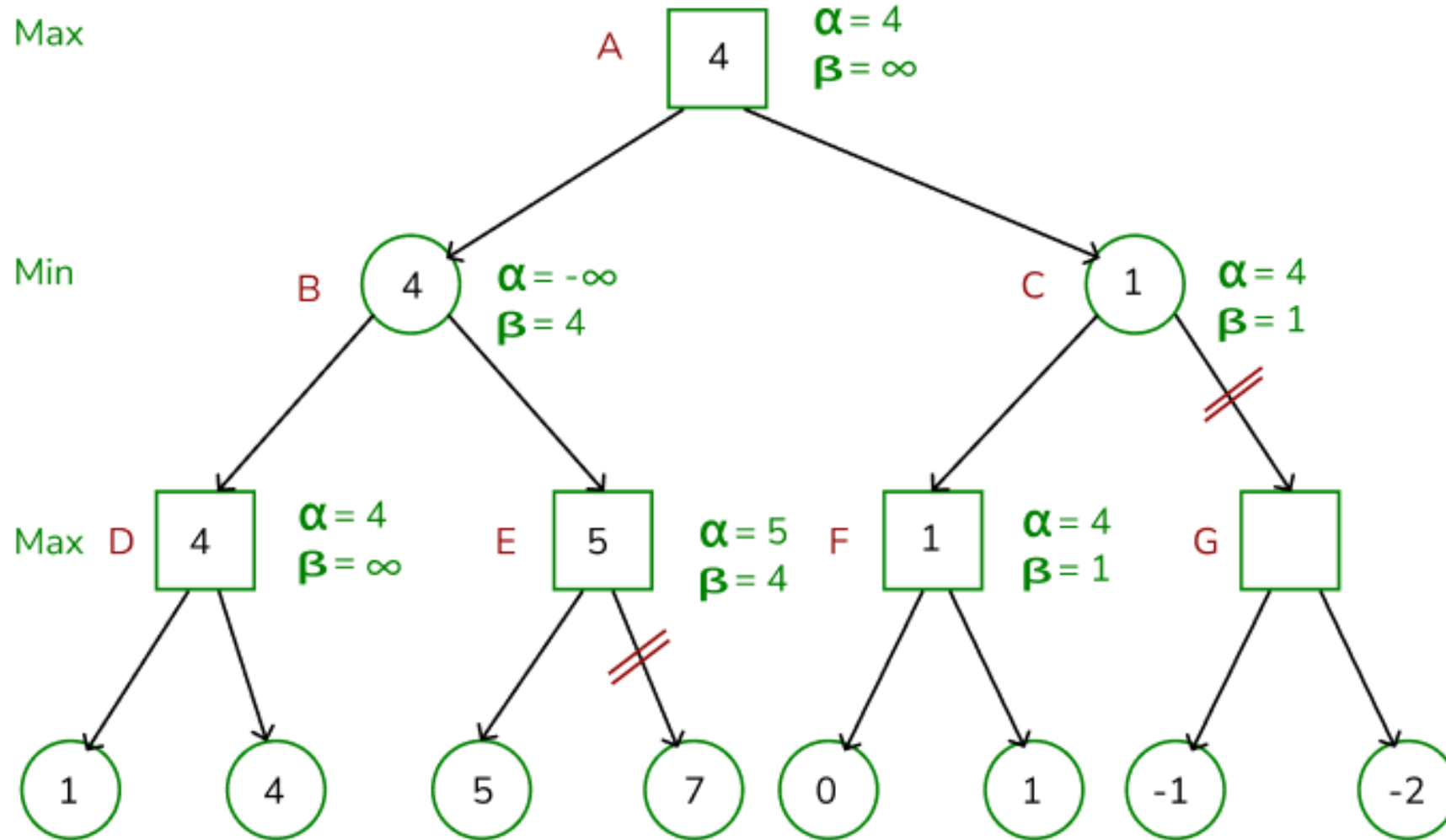
6

Repeat for all branches and finalize at root: Repeat steps 2–5 across the tree. The value that reaches the root (A) represents the optimal outcome, found faster since some branches were skipped.









Example of Alpha-Beta Pruning

Now that we've covered Alpha-Beta Pruning, let's move towards practical code-based example

Alpha-Beta Pruning

Alpha-Beta Pruning conditons



For Max-turn

```
    if best > alpha:
        alpha = best
    if beta <= alpha: # prune
        print(f"Pruning at {child} with alpha={alpha} and beta={beta}")
        break
```

For Min-turn

```
    if best < beta:
        beta = best
    if beta <= alpha: # prune
        print(f"Pruning at {child} with alpha={alpha} and beta={beta}")
        break
```

Lab Task: Minimax Algorithm & Alpha-Beta Pruning (Adversarial Search)

- *Implement the Minimax algorithm for Tic-Tac-Toe, where the AI evaluates all possible moves and selects the one that guarantees the best outcome, assuming the opponent also plays optimally.*
- *Enhance the Tic-Tac-Toe implementation from Task 1 by adding Alpha-Beta pruning to optimize the Minimax search. Compare the number of nodes explored with and without pruning, and show how alpha and beta values change at each step to eliminate unnecessary branches.*

Thank You

Questions or discussions are welcomed!